

Phase 1 — API Fetching

Goal:

- Connect Express backend with external NVD API using Axios.

What students learn:

- API integration
- Axios GET request
- Understanding API response structure

Flow:

```
Express Route
  ↓
Axios API Call
  ↓
response.data
```

Phase 2 — Nested Data Extraction & Transformation

Goal:

- Extract important fields from deeply nested JSON.

What students learn:

- Optional chaining (?.)
- find()
- map()
- flatMap()
- Navigating nested objects and arrays

Important concept:

- APIs return transport data
- Backend transforms data into database-friendly structure

Example:

```
Nested API JSON
  ↓
Extract important fields
  ↓
Create clean object
```

Phase 3 — Store One CVE in MongoDB

Goal:

- Save one transformed CVE document using Mongoose.

What students learn:

- Mongoose model usage
- create()
- Data flow from API → Database

Flow:

```
External API
    ↓
Transform data
    ↓
Store in MongoDB
```

Phase 4 — Store Multiple CVEs + Avoid Duplicates**Goal:**

- Process all vulnerabilities and prevent duplicate records.

What students learn:

- Looping through API data
- for...of
- updateOne()
- upsert

Important concept:

If document exists → update

If not exists → insert

Why important:

- Real-world sync systems
- ETL pipelines
- API ingestion workflows

Final flow:

```
Fetch API
    ↓
Loop vulnerabilities
    ↓
Transform data
    ↓
Upsert into MongoDB
```



PHASE 1 — Create Route

Create:

routes/cveRoutes.js

This route will:

- call NVD API
- receive response
- loop vulnerabilities

PHASE 2 — Understand & Extract Nested Data

Goal

Learn how to:

- Understand nested API response structure
- Extract important fields
- Transform complex JSON into clean objects

At this stage:

- No MongoDB storage yet
- No bulk insertion yet

Focus only on:

- Reading nested JSON
- Extracting useful data

Step 1 — Understand the API Structure

From Axios:

```
const response = await axios.get(URL);
```

The actual API response data is inside:

```
response.data
```

Structure:

```
response.data
├── vulnerabilities[]
│   └── cve
```

Extract vulnerabilities array:

```
const vulnerabilities = response.data.vulnerabilities;
```

Each item contains:

```
vulnerabilities[0].cve
```

Step 2 — Extract First CVE Object

```
const firstItem = vulnerabilities[0];
```

```
const cve = firstItem.cve;

console.log(cve);
```

At this point:

- cve contains one vulnerability object
- All nested fields exist inside this object

Step 3 — Extract Basic Fields

These are direct properties.

```
const cveId = cve.id;

const sourceIdentifier =
  cve.sourceIdentifier;

const published =
  cve.published;

const lastModified =
  cve.lastModified;

const vulnStatus =
  cve.vulnStatus;
Print extracted values:
console.log({
  cveId,
  sourceIdentifier,
  published,
  lastModified,
  vulnStatus
});
```

Step 4 — Extract Description

API structure:

```
"descriptions": [
  {
    "lang": "en",
    "value": "...
  }
]
```

Goal:

- Store only English description

Extraction:

```
const description =
  cve.descriptions?.find(
    d => d.lang === "en"
 )?.value || "";
```

Explanation:

Part	Purpose
find()	Finds English object
?.value	Prevents crash if missing
,	

Print:

```
console.log(description);
```

Step 5 – Extract Metrics

API structure:

```
metrics
├── cvssMetricV2[]
│   └── cvssData
```

Extract first metric object:

```
const metric =
  cve.metrics?.cvssMetricV2?.[0];
Extract important fields:
const score =
  metric?.cvssData?.baseScore;

const severity =
  metric?.baseSeverity;

const attackVector =
  metric?.cvssData?.accessVector;

const attackComplexity =
  metric?.cvssData?.accessComplexity;

const exploitabilityScore =
  metric?.exploitabilityScore;

const impactScore =
  metric?.impactScore;
Print:
console.log({
  score,
  severity,
  attackVector,
```

```
    attackComplexity,  
    exploitabilityScore,  
    impactScore  
  });
```

Step 6 — Extract Weaknesses

API structure:

```
weaknesses[]  
  description[]
```

Extraction:

```
const weaknesses =  
  cve.weaknesses?.flatMap(  
    weakness =>  
      weakness.description.map(  
        d => d.value  
      )  
  ) || [];
```

Output example:

```
[  
  "NVD-CWE-Other"  
]
```

Why flatMap()?

Without flatMap():

```
[  
  ["NVD-CWE-Other"]  
]
```

With flatMap():

```
[  
  "NVD-CWE-Other"  
]
```

Step 7 — Extract References

API structure:

```
references[]
```

Extract URLs:

```
const references =  
  cve.references?.map(  
    url => url.value
```

```
    ref => ref.url  
  ) || [];  
}
```

Output example:

```
[  
  "http://example.com",  
  "http://example2.com"  
]
```

Step 8 — Extract CPEs

API structure:

```
configurations[]  
  nodes[]  
    cpeMatch[]  
      criteria
```

Extraction:

```
const cpes = [];  
  
cve.configurations?.forEach(config => {  
  
  config.nodes?.forEach(node => {  
  
    node.cpeMatch?.forEach(match => {  
  
      cpes.push(match.criteria);  
  
    });  
  
  });  
  
});
```

Output example:

```
[  
  "cpe:2.3:a:example",  
  "cpe:2.3:o:example"  
]
```

Step 9 — Build Final Transformed Object

Now combine all extracted fields into a clean object.

```
const transformed = {
  cveId,
  sourceIdentifier,
  published,
  lastModified,
  vulnStatus,

  description,

  severity,
  score,

  attackVector,
  attackComplexity,

  exploitabilityScore,
  impactScore,

  weaknesses,
  references,
  cpes
};
```

Print final object:

```
console.log(transformed);
```

Important Learning Outcome

This phase teaches:

- Understanding nested API structures
- Optional chaining
- find()
- map()
- flatMap()
- Data transformation
- Converting transport data into database-friendly structure

Key Backend Engineering Principle

Do not directly store external API response.

Instead:



1. Extract important fields
2. Flatten nested structures
3. Transform data
4. Create query-friendly schema structure

PHASE 3 — Store One CVE Document in MongoDB

Goal

In this phase we will:

- Import the CVE model
- Fetch API data
- Transform one CVE object
- Save one document into MongoDB

At this stage:

- No bulk insertion yet
- No pagination yet
- No optimization yet

Focus only on:

- Storing one clean document successfully

Step 1 — Import CVE Model

routes/cveRoutes.js

Add:

```
import CVE from "../models/CVE.js";
```

Step 2 — Fetch API Data

```
const response = await axios.get(
  "https://services.nvd.nist.gov/rest/json/cves/2.0"
);

const vulnerabilities =
  response.data.vulnerabilities;
```

Step 3 — Get First CVE Object

```
const firstItem = vulnerabilities[0];

const cve = firstItem.cve;
```

Step 4 — Extract Basic Fields

```
const cveId = cve.id;

const sourceIdentifier =
  cve.sourceIdentifier;
```

```
const published =
  cve.published;

const lastModified =
  cve.lastModified;

const vulnStatus =
  cve.vulnStatus;
```

Step 5 — Extract Description

```
const description =
  cve.descriptions?.find(
    d => d.lang === "en"
 )?.value || "";
```

Step 6 — Extract Metrics

```
const metric =
  cve.metrics?.cvssMetricV2?.[0];

const score =
  metric?.cvssData?.baseScore;

const severity =
  metric?.baseSeverity;

const attackVector =
  metric?.cvssData?.accessVector;

const attackComplexity =
  metric?.cvssData?.accessComplexity;

const exploitabilityScore =
  metric?.exploitabilityScore;

const impactScore =
  metric?.impactScore;
```

Step 7 — Extract Weaknesses

```
const weaknesses =
```

```
cve.weaknesses?.flatMap(  
  weakness =>  
    weakness.description.map(  
      d => d.value  
    )  
  ) || [];
```

Step 8 — Extract References

```
const references =  
  cve.references?.map(  
    ref => ref.url  
  ) || [];
```

Step 9 — Extract CPEs

```
const cpes = [];  
  
cve.configurations?.forEach(config => {  
  
  config.nodes?.forEach(node => {  
  
    node.cpeMatch?.forEach(match => {  
  
      cpes.push(match.criteria);  
  
    });  
  
  });  
  
});
```

Step 10 — Create Transformed Object

```
const transformed = {  
  cveId,  
  sourceIdentifier,  
  published,  
  lastModified,  
  vulnStatus,  
  
  description,
```

```
severity,
score,

attackVector,
attackComplexity,

exploitabilityScore,
impactScore,

weaknesses,
references,
cpes,

raw: cve
};
```

Step 11 — Save into MongoDB

```
const savedCVE = await CVE.create(
  transformed
);
```

Step 12 — Return Response

```
res.json(savedCVE);
```

Final Route

routes/cveRoutes.js

```
import express from "express";
import axios from "axios";
import CVE from "../models/CVE.js";

const router = express.Router();

/*
=====
FETCH AND STORE ONE CVE
=====
*/
router.get("/sync", async (req, res) => {
  try {
    /*
=====
```

```
FETCH API
=====
*/

const response = await axios.get(
  "https://services.nvd.nist.gov/rest/json/cves/2.0"
);

const vulnerabilities =
  response.data.vulnerabilities;

/*
=====
FIRST CVE
=====
*/

const firstItem = vulnerabilities[0];

const cve = firstItem.cve;

/*
=====
BASIC FIELDS
=====
*/

const cveId = cve.id;

const sourceIdentifier =
  cve.sourceIdentifier;

const published =
  cve.published;

const lastModified =
  cve.lastModified;

const vulnStatus =
  cve.vulnStatus;

/*
=====
DESCRIPTION
=====
*/

const description =
  cve.descriptions?.find(
    d => d.lang === "en"
```

```

   )?.value || "";

/*
=====
METRICS
=====
*/

const metric =
  cve.metrics?.cvssMetricV2?.[0];

const score =
  metric?.cvssData?.baseScore;

const severity =
  metric?.baseSeverity;

const attackVector =
  metric?.cvssData?.accessVector;

const attackComplexity =
  metric?.cvssData?.accessComplexity;

const exploitabilityScore =
  metric?.exploitabilityScore;

const impactScore =
  metric?.impactScore;

/*
=====
WEAKNESSES
=====
*/

const weaknesses =
  cve.weaknesses?.flatMap(
    weakness =>
      weakness.description.map(
        d => d.value
      )
  ) || [];

/*
=====
REFERENCES
=====
*/

const references =

```

```
cve.references?.map(
  ref => ref.url
) || [];

/*
=====
CPES
=====
*/

const cpes = [];

cve.configurations?.forEach(config => {

  config.nodes?.forEach(node => {

    node.cpeMatch?.forEach(match => {

      cpes.push(match.criteria);

    });

  });

});

/*
=====
TRANSFORMED OBJECT
=====
*/

const transformed = {
  cveId,
  sourceIdentifier,
  published,
  lastModified,
  vulnStatus,

  description,

  severity,
  score,

  attackVector,
  attackComplexity,

  exploitabilityScore,
  impactScore,
```

```
        weaknesses,
        references,
        cpes,

        raw: cve
    };

    /*
    =====
    STORE IN DATABASE
    =====
    */

    const savedCVE =
        await CVE.create(transformed);

    /*
    =====
    RESPONSE
    =====
    */

    res.json(savedCVE);

} catch (err) {

    res.status(500).json({
        message: err.message
    });

}
});

export default router;
```

Expected Result

When you call:

GET /api/cves/sync

One CVE document will be:

- transformed
- stored in MongoDB
- returned in response

Important Learning Outcome

This phase teaches:

- API integration
- Nested data extraction
- Data transformation
- Mongoose document creation
- Backend data flow

Current Flow

```
External API
  ↓
Fetch JSON
  ↓
Extract fields
  ↓
Transform structure
  ↓
Store in MongoDB
```

PHASE 4 — Store Multiple CVEs + Avoid Duplicates

Goal

In this phase we will:

- Loop through all vulnerabilities
- Transform each CVE
- Store multiple documents
- Prevent duplicate entries

At this stage:

- Still no optimization
- No bulkWrite yet
- No pagination yet

Focus on:

- Iteration
- Transformation
- Duplicate prevention

Problem with Current Implementation

Current flow:

Fetch API



Store only first CVE

But API returns:

vulnerabilities[]

which contains:

- many CVE records

We now need to:

- iterate all records
- store each one

Step 1 — Loop Through Vulnerabilities

Replace:

```
const firstItem = vulnerabilities[0];
```

with:

```
for (const item of vulnerabilities) {  
  
  const cve = item.cve;
```

```
}
```

Now each iteration processes:

- one CVE object

Step 2 — Move Transformation Logic Inside Loop

Everything related to:

- extraction
- transformation
- storage

must happen inside loop.

Structure

```
for (const item of vulnerabilities) {  
  
  const cve = item.cve;  
  
  // extract fields  
  
  // create transformed object  
  
  // store in DB  
  
}
```

Step 3 — Duplicate Problem

If you call:

GET /api/cves/sync

multiple times:

same CVEs will try inserting again.

But:

cveId

is unique.

So MongoDB throws:

duplicate key error

Step 4 — Correct Solution: Upsert

Instead of:

create()

Use:

updateOne()

with:

upsert: true

Meaning:

If exists -> update

If not exists -> insert

Step 5 — Replace create()

Replace:

```
await CVE.create(transformed);
```

with:

```
await CVE.updateOne(  
  {  
    cveId: transformed.cveId  
  },  
  {  
    $set: transformed  
  },  
  {  
    upsert: true  
  }  
);
```

Step 6 — Add Counters

Useful for monitoring.

Add:

let processed = 0;

Inside loop:

processed++;

Step 7 — Final Response

Instead of returning one document:

```
res.json({  
  totalFetched: vulnerabilities.length,  
  processed  
});
```

Final Route

routes/cveRoutes.js

```
import express from "express";
import axios from "axios";
import CVE from "../models/CVE.js";

const router = express.Router();

/*
=====
FETCH AND STORE MULTIPLE CVEs
=====
*/
router.get("/sync", async (req, res) => {
  try {

    /*
    =====
    FETCH API
    =====
    */

    const response = await axios.get(
      "https://services.nvd.nist.gov/rest/json/cves/2.0"
    );

    const vulnerabilities =
      response.data.vulnerabilities;

    /*
    =====
    COUNTER
    =====
    */

    let processed = 0;

    /*
    =====
    LOOP ALL CVEs
    =====
    */

    for (const item of vulnerabilities) {

      const cve = item.cve;

      /*
      =====
      */
    }
  }
});
```

```
BASIC FIELDS
=====
*/

const cveId = cve.id;

const sourceIdentifier =
  cve.sourceIdentifier;

const published =
  cve.published;

const lastModified =
  cve.lastModified;

const vulnStatus =
  cve.vulnStatus;

/*
=====
DESCRIPTION
=====
*/

const description =
  cve.descriptions?.find(
    d => d.lang === "en"
 )?.value || "";

/*
=====
METRICS
=====
*/

const metric =
  cve.metrics?.cvssMetricV2?.[0];

const score =
  metric?.cvssData?.baseScore;

const severity =
  metric?.baseSeverity;

const attackVector =
  metric?.cvssData?.accessVector;

const attackComplexity =
  metric?.cvssData?.accessComplexity;
```

```

const exploitabilityScore =
  metric?.exploitabilityScore;

const impactScore =
  metric?.impactScore;

/*
=====
WEAKNESSES
=====
*/

const weaknesses =
  cve.weaknesses?.flatMap(
    weakness =>
      weakness.description.map(
        d => d.value
      )
  ) || [];

/*
=====
REFERENCES
=====
*/

const references =
  cve.references?.map(
    ref => ref.url
  ) || [];

/*
=====
CPES
=====
*/

const cpes = [];

cve.configurations?.forEach(config => {

  config.nodes?.forEach(node => {

    node.cpeMatch?.forEach(match => {

      cpes.push(match.criteria);

    });

  });

});

```

```
});

/*
=====
TRANSFORMED OBJECT
=====
*/

const transformed = {
  cveId,
  sourceIdentifier,
  published,
  lastModified,
  vulnStatus,

  description,

  severity,
  score,

  attackVector,
  attackComplexity,

  exploitabilityScore,
  impactScore,

  weaknesses,
  references,
  cpes,

  raw: cve
};

/*
=====
UPSERT
=====
*/

await CVE.updateOne(
  {
    cveId: transformed.cveId
  },
  {
    $set: transformed
  },
  {
    upsert: true
  }
);
```

```
    );

    processed++;

  }

  /*
  =====
  RESPONSE
  =====
  */

  res.json({
    totalFetched: vulnerabilities.length,
    processed
  });

} catch (err) {

  res.status(500).json({
    message: err.message
  });

}
});

export default router;
```

Important Learning Outcome

This phase teaches:

- Iterating API data
- Processing arrays of objects
- Upsert operation
- Duplicate prevention
- Incremental synchronization

Important Backend Concept

Why Upsert is Better Than create()

create()

Insert only

Fails on duplicates

upsert

Update if exists

Insert if missing

Much better for:

- sync systems
- ETL pipelines
- API ingestion systems

Current Backend Flow

```
External API
  ↓
Loop vulnerabilities
  ↓
Extract nested fields
  ↓
Transform structure
  ↓
Upsert into MongoDB
```